

CS-600: Software Design and Development

8-1 Short Paper: Life Cycle Thinking in Software Design

Malgorzata Debska

Southern New Hampshire University

Matthew Scuteri

June 8, 2026

Sustainability Plan and Test Strategy for Application Enhancement

As software applications grow and new features are added, it becomes important to ensure that the system is still reliable, secure, and easy to keep. A sustainability plan helps organizations prepare for future growth while reducing the risk of performance issues, security problems, and costly maintenance. The roadmap for the current application includes several updates that will transform it from a demonstration application into a fully functional product. These updates include database-based user authentication, user management features, record editing capabilities, search and sorting functions, and audit logging. To support these enhancements, the development team must create a sustainability plan and a test plan that ensures the application continues to perform effectively as it evolves.

Roadmap Areas That Impact Sustainability

Several items on the roadmap have a direct impact on the long-term sustainability of the application. One of the most important updates is replacing hard-coded usernames and passwords with credentials stored in a database using encrypted passwords. This change improves security and makes user management easier because administrators can update accounts without changing application code. The roadmap also includes new functionality that allows administrators to create users, manage roles, edit records, cut records, search for information, and sort data. While these features improve usability, they also increase system

complexity. As more users and data are added over time, the application must continue to perform efficiently. Another significant addition is audit logging, which records who made changes and when those changes occurred. This feature supports accountability, troubleshooting, and compliance requirements. Because the application will handle more users, more data, and added security responsibilities, careful planning is necessary to ensure that it stays maintainable and reliable in the future.

Sustainability Plan

Service-Level Agreements (SLAs)

To support a positive user experience, the application should meet several service-level agreements. System availability should remain at 99.9% uptime each month. User login requests should be processed within two seconds, while search and sorting functions should return results within three seconds. Password changes should be completed within five seconds, and any critical system failures should be resolved within one hour. According to Chandrasekaran and Krishnan (2022), service-level agreements help organizations set up measurable goals that define expected system performance and reliability. These goals provide clear benchmarks for evaluating whether the application is meeting user expectations.

Key Performance Metrics

Several performance metrics should be checked continuously to measure system health and reliability. These include system uptime, average response times, database performance, authentication of success rates, failed login attempts, error rates, and recovery times following

system failures. Monitoring these metrics allows developers to find potential issues early and take corrective action before users are affected. Observability and performance monitoring are important aspects of sustainable software because they provide insight into how the system behaves under real-world conditions (Chandrasekaran & Krishnan, 2022).

Testing Methods

Long-term reliability requires a combination of testing methods throughout the software life cycle:

- Unit testing will verify that individual functions run correctly, including authentication, password management, search features, and record updates.
- Integration testing will confirm that the application communicates properly with the database and security components.
- Performance testing will evaluate how the application responds under different workloads and decides whether response time goals are being met.
- Security testing will verify that passwords are encrypted correctly, user permissions are enforced, and unauthorized users cannot access restricted functionality.
- Regression testing will be performed whenever changes are introduced to ensure that existing features continue to work correctly.
- Finally, user acceptance testing will allow stakeholders to confirm that the implemented features satisfy business and user requirements before deployment.

Using multiple testing methods creates a solid foundation for keeping software quality and reducing future maintenance costs.

Supporting Roadmap Execution

The sustainability plan directly supports successful implementation of the roadmap. Security-focused testing and monitoring help ensure that the transition to database-based authentication is still secure and reliable. Performance monitoring helps developers evaluate the impact of new search, sorting, and user management features as the system grows. The addition of audit logging also supports sustainability by providing a clear record of system activity. If problems occur, developers and administrators can quickly find out what changes were made and by whom. This improves troubleshooting and reduces downtime. By proving measurable goals and verification procedures, the sustainability plan helps ensure that roadmap features can be implemented without negatively affecting overall system performance.

Test Plan

The test plan links each system requirement to specific verification activities. The database authentication requirement will be verified through integration and security testing to ensure users can log in using encrypted credentials. User creation and role management features will be verified through functional testing and security testing to confirm that permissions are assigned correctly. Password change functionality will be confirmed through functional testing to ensure users can update credentials successfully. Record editing and deletion features will be verified through functional and regression testing to ensure only authorized users can change data. Search and sorting capabilities will be evaluated through both functional and performance testing to confirm that correct results are returned within established response-time goals.

Finally, audit logging will be verified through integration testing to ensure that all changes are recorded with the correct user information and timestamps. In addition to verifying functionality, testing will confirm sustainability goals by measuring uptime, response times, error rates, and recovery performance. Continuous monitoring of these metrics will show whether the application continues to meet established service levels over time.

Professional Best Practices

The sustainability and test plans support several professional software engineering best practices. First, they encourage proactive monitoring instead of reacting to problems after they occur. Second, they support continuous testing, which helps name defects early in development. Third, they promote maintainability by ensuring that future updates can be implemented without disrupting existing functionality. These practices also help reduce technical debt and improve overall software quality. Pressman and Maxim (2020) explain that successful software projects require ongoing verification, maintenance planning, and performance monitoring throughout the system life cycle. By combining sustainability planning with comprehensive testing, organizations can improve reliability while supporting future growth.

Conclusion

The roadmap introduces several important enhancements that will transform the application into a secure and fully functional product. However, these new capabilities also create more responsibilities related to security, performance, and maintenance. The sustainability plan provides clear service-level goals, performance metrics, and testing strategies that support long-

term reliability. The test plan ensures that each requirement is properly verified, and that sustainability aims continue to be met as the application evolves. Together, these plans provide a structured approach for supporting future development while keeping software quality and user satisfaction.

References

Chandrasekaran, P., & Krishnan, K. (2022). *Domain-Driven Design with Java: A Practitioner's Guide*. Packt Publishing.

Pressman, R. S., & Maxim, B. R. (2020). *Software Engineering: A Practitioner's Approach* (9th ed.). McGraw-Hill Education.